COLLEGE OF COMPUTING AND INFORMATICS
PUTRAJAYA CAMPUS

SEMESTER 2 2025/2026

SOFTWARE CONSTRUCTION & METHODS (CSEB5223)

LAB 3 AND LAB 4 ASSIGNMENT

SEMESTER 2 2025/2026

PROGRAMME : BACHELOR OF SOFTWARE ENGINEERING (HONS.)
GROUP ASSIGNMENT : GROUP 6
PROJECT TITLE : STUDENT LEARNING MANAGEMENT SYSTEM (SLMS)
LECTURER NAME : TS. DR. LOO YIM LING
GROUP MEMBERS :

No.	Full Name	Student ID
1	ARIFAH NUR IMAN BINTI MOHD FADZILAH	BSW01085214
2	REYHANA MARSYA BINTI AHMAD SAIFULLIZAN	BSW01085069
3	DESHINI A/P SELVAKUMAR	SW01083593
4	PATVIR SINGH KHOSA	BSW01085393

GITHUB REPOSITORY URL: https://github.com/Gwigle/SLMS-CSEB5223-Group6.git

Table of Contents

1.0 Introduction 3

2.0 Use Case Design 4

2.1 Use Case 4

2.2 Use Case Diagram 5

3.0 Class Design 6

3.1 Class Relationships 7

3.2 UML Class Diagram 9

4.0 Documentation Formatting 10

5.0 Coding File Formatting 10

6.0 Naming Conventions – Dictionary 11

6.2 Naming Dictionary 12

7.0 Function and Methods Modularity 14

7.1 Principles of Modularity Applied in SLMS 14

7.2 Method-Level Modularity 14

7.3 Reusability of Methods 15

7.4 Low Coupling and High Cohesion 16

7.5 Summary of Modularity Benefits in SLMS 16

8.0 Continuous Integration Rules 17

9.0 Course Profile 19

9.1 Class Design 19

9.2 Course Input Implementation 23

9.3 Search Function Development 26

9.4 Edit Function Development 28

9.5 Delete Function Development 30

9.6 Data Display Function Development 33

10.0 References 36

1.0 Introduction

The Student Learning Management System (SLMS) is a console-based Java application designed to allow administrators to efficiently manage students, courses and their relationships in a centralized system. The system implements a menu-driven interface, enabling easy input, search, editing, deletion and display of student and course profiles.

In Lab 2, the focus was on drafting the system plan, including use case design, class design, documentation, coding standards, naming conventions, modular functions and continuous integration practices. This established a strong foundation for structured software construction.

In Lab 3, we implemented course input, search, and display functionalities. Users can add courses with attributes such as course code, name, credit hours, summary, MS Teams link and type (core or elective). Input validation ensures data integrity, while the displayCourse() method provides a clear, formatted console output.

In Lab 4, the system was extended to support editing, deletion and viewing of all course profiles, with robust error handling for invalid inputs and prevention of duplicate entries. The system now maintains an array-based storage of courses and students, with a relationship matrix to track course-student assignments.

Overall, the SLMS leverages Object-Oriented Programming (OOP) principles through encapsulation, modular design and reusable methods. The array-based storage, combined with input validation and error handling, ensures system robustness, while GitHub version control supports collaborative development and continuous integration

2.0 Use Case Design

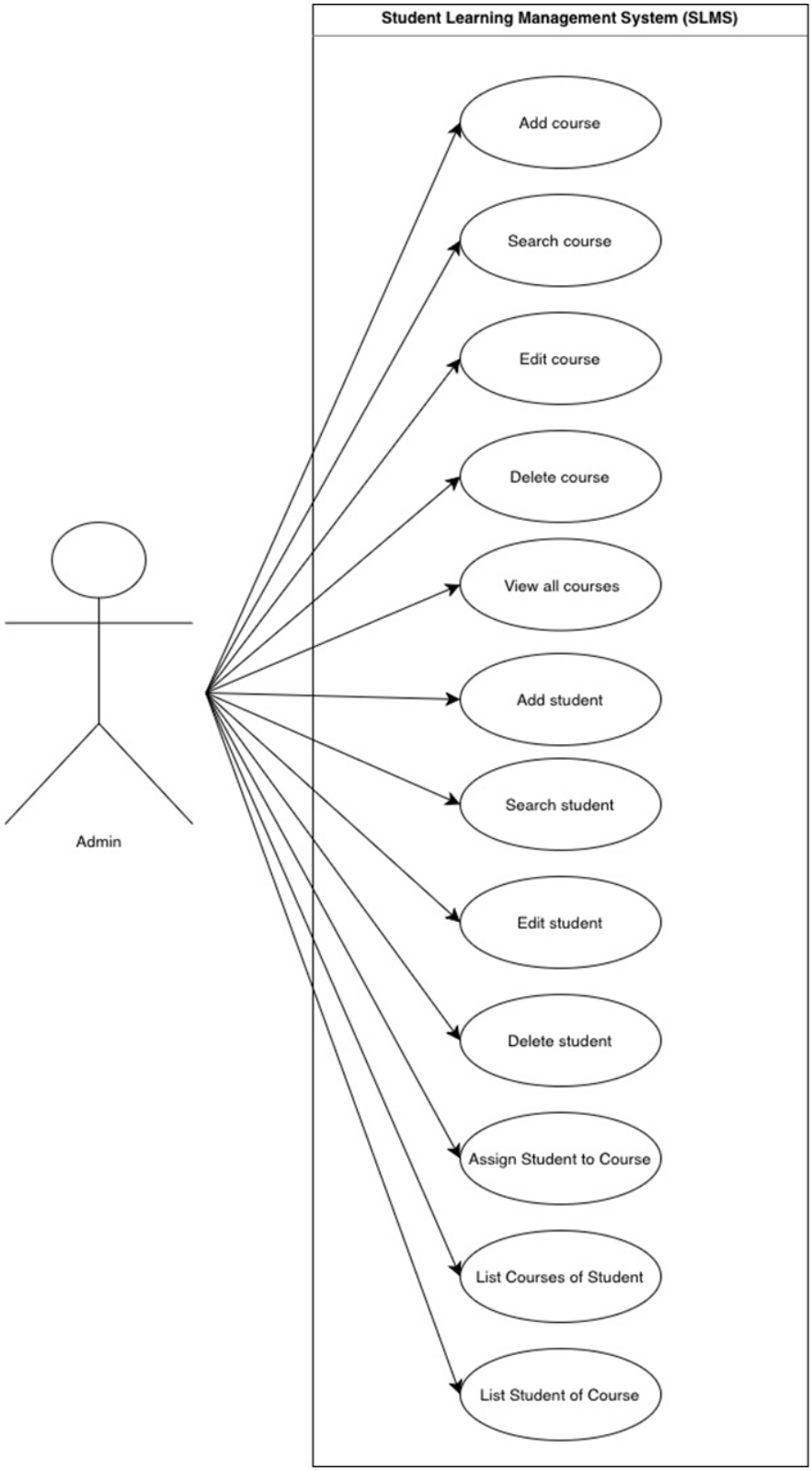
The Use Case Design describes how users interact with the Student Learning Management System (SLMS). It identifies the system of actor and the different actions that can be performed within the system. In SLMS, the main actor is the Admin, who is responsible for managing courses and student information.

The use cases illustrate the functionalities available in the system such as adding, searching, editing and deleting courses and students, as well as assigning students to courses. These interactions help to define the system requirements clearly and provide a visual understanding of how the system operates.

2.1 Use Case

Actor	Use Case	Description
Admin	Add course	Admin inputs course information and adds it to the system.
Admin	Search course	Admin searches for a course by course code.
Admin	Edit course	Admin edits any course attributes except course code.
Admin	Delete course	Admin deletes a course after confirmation; removes relationships.
Admin	View all courses	Admin views all courses with complete details.
Admin	Add student	Admin inputs student information and adds it to the system.
Admin	Search student	Admin searches for a student by student ID.
Admin	Edit student	Admin edits student attributes except student ID.
Admin	Delete student	Admin deletes a student after confirmation; removes relationships.
Admin	Assign Student to Course	Admin assigns a student to a course.
Admin	List Courses of Student	Admin views all courses a student is enrolled in.
Admin	List Students of Course	Admin views all students enrolled in a course.

2.2 Use Case Diagram



3.0 Class Design

The Class Design section provides a detailed blueprint of the system's structure by defining the classes, their attributes, methods, and the relationships between them. In the Student Learning Management System (SLMS), the class design follows object-oriented programming principles to ensure modularity, reusability and maintainability.

Class	**Attribute**	**Method**
Course	courseCode: String courseName: String creditHour: int summary: String teamsLink: String courseType: String	getCourseCode( ) getCourseName( ) setCourseName( ) getCreditHour( ) setCreditHour( ) getSummary( ) setSummary( ) getTeamsLink( ) setTeamsLink( ) getCourseType( ) setCourseType( ) displayCourse( )
Student	studentID: String studentName: String email: String phone: String	getStudentID( ) getStudentName( ) setStudentName( ) getEmail( ) setEmail( ) getPhone( ) setPhone( ) displayStudent( )
SLMSManager	courses: Course[ ] students: Student[ ]	addCourse( ) searchCourse( )

	relationship: boolean[ ][ ] courseCount: int studentCount: int	editCourse( ) deleteCourse( ) viewAllCourses( ) addStudent( ) searchStudent( ) editStudent( ) deleteStudent( ) viewAllStudents( ) assignStudentToCourse( ) listCoursesOfStudent( ) listStudentsOfCourse( )
Main	None	main(String[ ] args)

3.1 Class Relationships

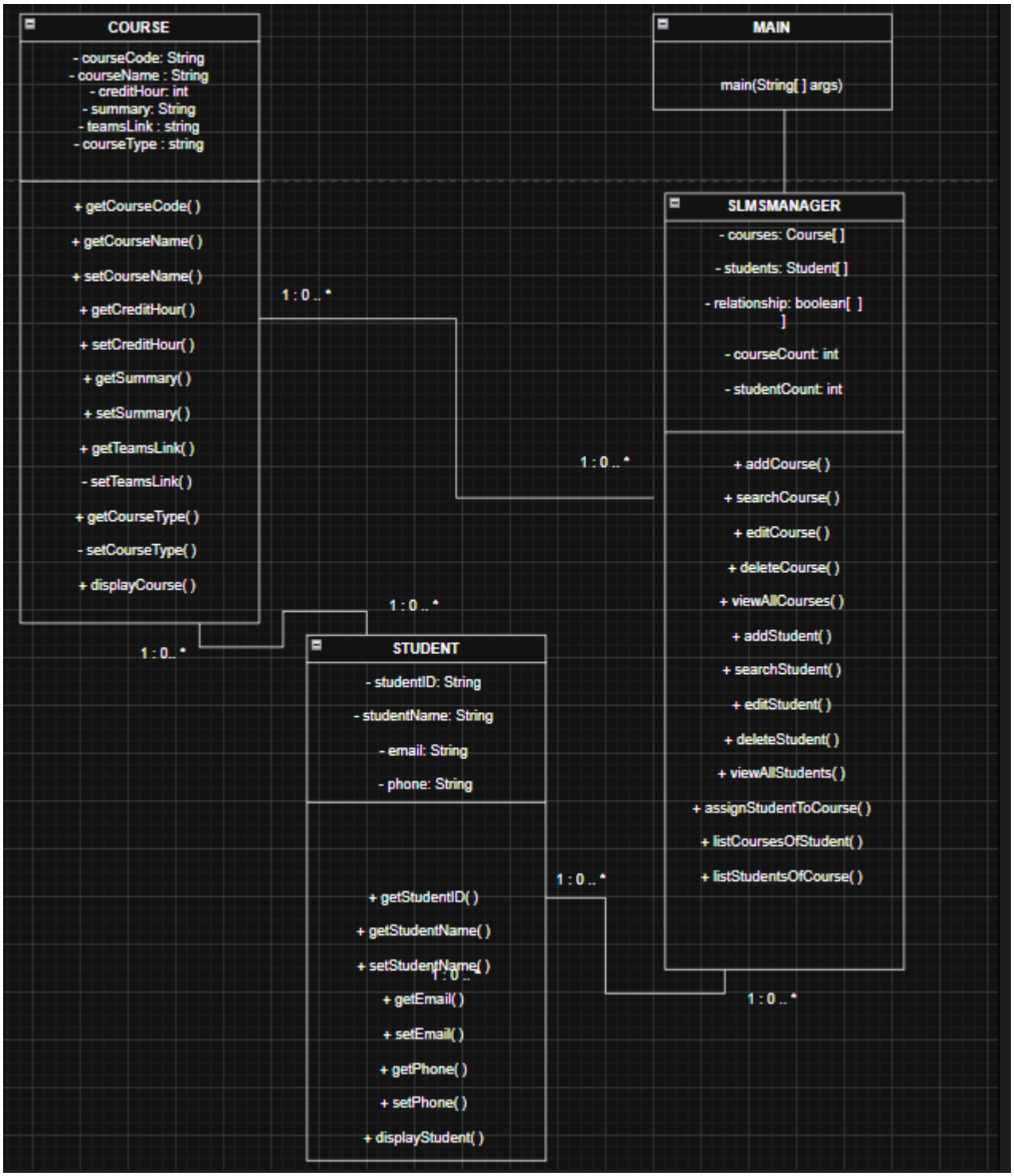
Association Relationships

- Association represents a structural relationship between different classes, showing how instances of one class know about instances of another. In our SLMS, these are primarily "has-a" or "manages" relationships, managed by the SLMSManager class.

Class A	Relationship	Class B	Multiplicity	Description
SLMS Manager	manages	Course	1:0..*	One SLMSManager can manage zero or many Course objects, stored in the courses array.
SLMSManager	manages	Student	1:0..*	One SLMSManager can manage zero or many Student objects, stored in the students array.

SLMSManager	assigns	Student	1:0..*	The SLMSManager assigns a Student to a Course, as recorded in the relationship matrix.
Student	Enrolled in	Course	1:0..*	A Student can be enrolled in zero or many Course objects, tracked via the manager's relationship matrix.
Course	Has enrolled	Student	1:0..*	A Course can have zero or many enrolled Student objects, tracked via the manager's relationship matrix.

3.2 UML Class Diagram



4.0 Documentation Formatting

- Font: Times New Roman, 12
- Headings: Bold, Times New Roman, 12
- Paragraph: Justified
- Line spacing: 1.5

5.0 Coding File Formatting

To maintain consistency and readability, the following coding file formatting standards are applied in the SLMS project:

- Programming Language: Java
- File Naming: Each class is stored in a separate .java file.
- Indentation: 4 spaces indentation is used for code blocks.
- Brackets Style: Opening braces { are placed on the same line as the class or method declaration.
- Comments: Single-line comments use //, multi-line comments use /* */
- File Header: Every file must begin with a comment block containing the file name, our names and a brief description of the class.
- Line Length: Limit code lines to 120 characters to avoid horizontal scrolling.
- Imports: Group imports by package and avoid using wildcards (e.g., use import java.util.Scanner; instead of java.util.*;).
- White Space: Use a single blank line between methods and after variable declarations to improve scannability.

6.0 Naming Conventions – Dictionary

Naming conventions are a pre-defined set of rules and/or guidelines used to give elements, variables, class, and other types of code readable and consistent names/identifiers to ensure readability, consistency and maintainability.

6.1 Naming Conventions

Naming conventions help developers to write readable and consistent code by applying the set of rules and/or guidelines to denote identifiers.

Element	**Convention**	**Example**
Class	PascalCase	Course, Student, SLMSManager, Main
Method	camelCase	addCourse(), searchCourse(), editCourse(), deleteCourse(), viewAllCourses()
Variable	camelCase	courseCode, courseName, creditHour, summary, teamsLink, courseType
Constant	UPPER_CASE	MAX_COURSES
File Name	Same as class name	Course.java
Boolean Variable	Starts with is/has	isEnrolled, hasSubmitted

6.2 Naming Dictionary

A naming dictionary acts as a glossary to define naming conventions and identify what type of code it is.

Name	Type	Description
Course	Class	Stores course information
Student	Class	Represents a student in the system
SLMSManager	Class	Represents a SLMS manager in the system
Main	Class	Represents the whole system
getCourseCode()	Method	Allows user to get course code
getCourseName()	Method	Allows user to get course name
setCourseName()	Method	Allows admin to set course name
getCreditHour()	Method	Allows user to get credit hour(s)
setCreditHour()	Method	Allows admin to set credit hour(s)
getSummary()	Method	Allows user to get summary
setSummary()	Method	Allows admin to set summary
getTeamsLink()	Method	Allows user to get the URL link to Teams
setTeamsLink()	Method	Allows admin to set the URL to Teams
getCourseType()	Method	Allows user to see what type the course is that they are taking
setCourseType()	Method	Allows admin to set what type the course is
displayCourse()	Method	Display course information
getStudentID()	Method	Allows user to get student ID
getStudentName()	Method	Allows user to get the name of students in the system
setStudentName()	Method	Allows admin to set the name of student into the system
getEmail()	Method	Allows user to get email of students in the system
setEmail()	Method	Allows admin to set the email of student into the system
getPhone()	Method	Allows user to get phone numbers in the systems
setPhone()	Method	Allows admin to set phone numbers into the system
displayStudent()	Method	Displays student information
addCourses()	Method	Allows the admin to add a course into the system
searchCourse()	Method	Allows admin to search for a course in the system

editCourse()	Method	Allows admin to edit a course in the system
deleteCourse()	Method	Allows admin to delete a course off the system
viewAllCourse()	Method	Allows admin to view all courses in the system
addStudent()	Method	Allows admin to add a student into the system
searchStudent()	Method	Allows admin to search for a student in the system
editStudent()	Method	Allows admin to edits attributes on a student in the system
deleteStudent()	Method	Allows admin to delete a student off the system
viewAllStudent()	Method	Allows admin to view all students in the system
assignStudentToCourse ()	Method	Allows admin to assign a student into any course
listCoursesOfStudents()	Method	Allows admin to see the list of courses that a student is assigned to.
listStudentsOfCourse()	Method	Allows admin to see the list of students that are assigned to a course

7.0 Function and Methods Modularity

Modularity in software construction and methods refers to dividing a system into separate, independent modules where each module contains everything needed to execute a specific functionality. In the Student Learning Management System (SLMS), we apply modularity principles to ensure our methods are reusable and maintainable.

7.1 Principles of Modularity Applied in SLMS

1. **Single Responsibility Principle (SRP)**

- Each class and method performs only one clear responsibility. Example:
  - getCourseCode() in Course class only handles in getting course codes.
  - getStudentID() in Student class only handles getting Student IDs.
  - addCourse() in SLMSManager class only handles adding courses into the system.

2. **Separation of Concerns**

- Each class handles a specific part of the system:
  - User-related operations – SLMSManager, Student
  - Course management - Course

3. **Encapsulation**

- All attributes are declared as private and accessed using getter and setter methods.

7.2 Method-Level Modularity

- Each method is:
  - short and concise
  - named using CapitalCase
  - programmed to follow one specific instruction
  - can be reusable

7.3 Reusability of Methods

Reusability ensures that methods can be used multiple times throughout the system without rewriting the same code.

- Reduces code duplication in the system.
- Improves code maintainability and readability.
- Allows methods to be reused in multiple functions.

Examples in SLMS

- displayCourse()
  - Used to display course information.
  - Reused in features like *viewAllCourses()* and *searchCourse()*.
- displayStudent()
  - Displays student information.
  - Can be reused when viewing or searching for student records.
- Getter methods such as:
  - getCourseCode()
  - getCourseName()
  - getStudentID()

These methods allow other classes to access object data safely without directly modifying it.

Using reusable methods makes the SLMS system easier to maintain, update and expand in future development.

7.4 Low Coupling and High Cohesion

Low coupling and high cohesion help create a well-structured and maintainable software system.

High Cohesion

- Each class focuses on one specific responsibility.
- The Course class manages course information.
- The Student class manages student information.
- The SLMSManager class manages system operations such as adding, searching, editing and deleting data.

Low Coupling

- Classes do not depend heavily on each other.
- The Course and Student classes only store data and methods related to themselves.
- The SLMSManager class controls interactions between students and courses.

Benefits of applying these principles

- Improves system organization
- Makes debugging easier
- Supports future system expansion

7.5 Summary of Modularity Benefits in SLMS

A. Maintainability - Small, focused methods are easier to update and debug
B. Reusability - Utility methods and service classes prevent code duplication
C. Readability - Method names clearly indicate what each function does
D. Scalability - New features can be added by creating new methods without breaking existing ones
E. Collaboration - Team members can work on different methods simultaneously with minimal conflicts

8.0 Continuous Integration Rules

Continuous Integration (CI) is a software development practice that ensures code changes from all team members are regularly integrated, tested and merged into the main branch. In the SLMS project, CI helps maintain a stable, working codebase, detect issues early and streamline collaboration among students, lecturers and administrators.

Branching and Pull Requests

- The main branch is protected, and no direct commits are allowed.
- Developers work on feature or bugfix branches, named clearly to reflect their purpose.
- All code changes are submitted via Pull Requests (PRs), reviewed by at least one team member.
- The team leader approves and merges PRs, ensuring quality control.

Commit Message Standards

- Commit messages follow the format: [Module] Short Description
- Guidelines: present tense, concise, descriptive and avoid vague messages.
- Examples:
  - [Course] Add search function
  - [Student] Add profile editing function

Testing and Code Validation

- Code must compile and pass all local tests before submission.
- Unit tests ensure new features do not break existing modules.
- Automated Pull Request checks verify runtime stability and functionality.

Repository Structure

- Organized folders support modularity, maintainability, and collaboration.
- Documentation is kept up to date with each code addition.

```
/SLMS
    /src
        /course
        /student
        /assignment
    /tests
    /docs
    README.md
```

Collaboration Practices

- Team members must pull before pushing to prevent conflicts.
- Clear communication is required for changes via GitHub issues or project boards.
- Code reviews help share knowledge and enforce best practices.

9.0 Course Profile

The Course Profile module of SLMS is responsible for managing course data, including input, search, edit, delete and display functionalities.

9.1 Class Design

The Course class encapsulates all relevant attributes for a course and provides getter, setter and display methods.

File Header

The first section is a comment block by using multi-line comments /* */. It contains the file name which is Course.java, our names and a brief description of the Course class.

```
Course.java
Source History
/*
File:
Course.java

Author:
1. ARIFAH NUR IMAN BINTI MOHD FADZILAH
2. REYHANA MARSYA BINTI AHMAD SAIFULLIZAN
3. DESHINI A/P SELVAKUMAR
4. PATVIR SINGH KHOSA

Description:
Represents a course in the SLMS system with attributes, getters, setters and display functionality.
 */
```

Class Declaration

public class course{}: This defines a public class named Course. Being public means other parts of the program can create "Course" objects.

Attributes

These private variables store the specific data for each course. The access modifier “private” is a practice called encapsulation, which protects the data from being changed directly from outside the class.

Attribute	Type	Description	Example Input
courseCode	String	Unique identifier of the course.	“CSEB5223”
courseName	String	Full name of the course.	“Software Construstion & Method”
creditHour	int	Number of credit hours.	3
summary	String	Brief description of the course.	“Practice coding and build SLMS project.”
teamsLink	String	Microsoft Teams link for course resources.	"https://teams.microsoft.com/l/course/CSEB5223"
courseType	String	Type of course: core or elective.	“core”

```java
public class Course {

    private String courseCode;
    private String courseName;
    private int creditHour;
    private String summary;
    private String teamsLink;
    private String courseType; // core, elective
```

Constructor

The parameters take the inputs and use the “this” keyword to assign those inputs to the private variables we defined above.

```java

    public Course(String courseCode, String courseName, int creditHour,
                  String summary, String teamsLink, String courseType) {
        this.courseCode = courseCode;
        this.courseName = courseName;
        this.creditHour = creditHour;
        this.summary = summary;
        this.teamsLink = teamsLink;
        this.courseType = courseType;
    }
```

Getter and Setter Methods

Since the variables are private, other classes cannot use them. Hence, we use getter methods to allow other parts of the program to "get" the value of the course code, name, credit hour, summary, teams link and course type. Then, we use setter method to update the data after the course has already been created. There is no setter method for courseCode, so that a course ID cannot be changed once it is set.

```java

    // Getters
    public String getCourseCode() { return courseCode; }
    public String getCourseName() { return courseName; }
    public int getCreditHour() { return creditHour; }
    public String getSummary() { return summary; }
    public String getTeamsLink() { return teamsLink; }
    public String getCourseType() { return courseType; }

    // Setters
    public void setCourseName(String courseName) { this.courseName = courseName; }
    public void setCreditHour(int creditHour) { this.creditHour = creditHour; }
    public void setSummary(String summary) { this.summary = summary; }
    public void setTeamsLink(String teamsLink) { this.teamsLink = teamsLink; }
    public void setCourseType(String courseType) { this.courseType = courseType; }

```

Course Display

The displayCourse() method is used to display all course information stored in the Course object. It prints the course details in a structured format so users can easily read the information. This method uses System.out.println() and System.out.printf() to display the course attributes. Formatting symbols such as %s (String) and %d (integer) are used to ensure the output is neatly aligned. The separator lines help make the course information clearer when multiple courses are displayed.

```java
    // Display course information form
    public void displayCourse() {
        System.out.println("==================================");
        System.out.printf("Course Code : %s%n", courseCode);
        System.out.printf("Course Name : %s%n", courseName);
        System.out.printf("Credit Hour : %d%n", creditHour);
        System.out.printf("Summary     : %s%n", summary);
        System.out.printf("Teams Link  : %s%n", teamsLink);
        System.out.printf("Type        : %s%n", courseType);
        System.out.println("==================================");
    }
}
```

Course Array Initialization

The courses array is declared in the SLMSManager class to store course records within the system. This array is created with a size of 100, which means the system can store up to one hundred Course objects.

Each element in the courses array represents a course and can store information related to that course through the Course class. This structure allows the system to manage multiple course records efficiently during program execution.

The variable courseCount is initialized to 0 and is used to keep track of the number of courses currently stored in the system. Whenever a new course is added, the course information will be stored in the courses array, and the value of courseCount will be increased accordingly. This ensures the system always knows how many valid course records exist.

```
    private Course[] courses = new Course[100];
    private int courseCount = 0;
```

Sample Data Initialization

Sample course data is initialized in the SLMSManager() constructor so that the system already contains several course records when the program starts. In this constructor, several Course objects are created and stored in the courses array.

Each course contains the course code, course name, credit hour, summary, Teams link and course type. The variable courseCount is set to 3, indicating that three course records are currently stored in the system.

```
    // Sample data
    public SLMSManager() {
        courses[0] = new Course("CSEB5223", "Software Construction & Methods", 3,
                "Practice coding and build SLMS project.", "https://teams.microsoft.com/l/course/CSEB5223", "core");
        courses[1] = new Course("CSEB4113", "Software Quality", 3,
                "Learn software quality and testing.", "https://teams.microsoft.com/l/course/CSEB4113", "core");
        courses[2] = new Course("CCSB5113", "Software Security", 3,
                "Fundamentals of software security.", "https://teams.microsoft.com/l/course/CCSB5113", "elective");
        courseCount = 3;
```

9.2 Course Input Implementation

The Course Input Implementation allows the Admin to add new course information into the Student Learning Management System (SLMS) through console input.

Based on the lab requirements, the system must allow the Admin to enter course details and store them in the system. In our implementation, the submit functionality is handled by the addCourse() method in the SLMSManager class.

When the Admin selects the Add Course option from the menu, the system prompts the user to enter the required course information such as:

- Course Code
- Course Name
- Credit Hour
- Course Summary
- Microsoft Teams Link
- Course Type (Core or Elective)

The system then checks whether the course code already exists to prevent duplicate entries. If the course code is unique, a new Course object is created and stored in the courses array. The courseCount variable is then increased to keep track of the number of courses stored in the system. This system store multiple courses using an array and it also prevent array out-of-bound errors.

This implementation ensures that course information is properly stored and managed while maintaining data integrity in the SLMS.

Code

```java
    //course functions
    public void addCourse() {
        if(courseCount >= courses.length) {
            System.out.println("Course list full!");
            return;
        }

        System.out.print("Course Code: ");
        String code = sc.nextLine();
        if(findCourseIndex(code) != -1) {
            System.out.println("Course exists!");
            return;
        }

        System.out.print("Course Name: ");
        String name = sc.nextLine();

        int credit = 0;
        while(true) {
            try {
                System.out.print("Credit Hour: ");
                credit = Integer.parseInt(sc.nextLine());
                if(credit <= 0) throw new NumberFormatException();
                break;
            } catch(NumberFormatException e) {
                System.out.println("Invalid input! Please enter a positive integer for credit hour.");
            }
        }

        System.out.print("Summary: ");
        String summary = sc.nextLine();

        System.out.print("Teams Link: ");
        String link = sc.nextLine();
```

```java
    }
}

System.out.print("Summary: ");
String summary = sc.nextLine();

System.out.print("Teams Link: ");
String link = sc.nextLine();

String type = "";
while(true) {
    System.out.print("Course Type (core/elective/university): ");
    type = sc.nextLine().toLowerCase();
    if(type.equals("core") || type.equals("elective") || type.equals("university")) break;
    else System.out.println("Invalid course type. Please enter 'core', 'elective', or 'university'.");
}

courses[courseCount++] = new Course(code,name,credit,summary,link,type);
System.out.println("Course added successfully!");
```

Output

```
run:

===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice: 1
Course Code: CSEB6001
Course Name: Artificial Intelligence
Credit Hour: 3
Summary: Introduction to AI concepts
Teams Link: https://teams.microsoft.com/l/course/CSEB6001

Course Type (core/elective/university): Invalid course type. Please enter 'core', 'elective', or 'university'.
Course Type (core/elective/university): core
Course added successfully!

===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice:
```

9.3 Search Function Development

With the Search Function, the system allows the user to find a specific course by entering the course code. The program uses a linear search method to check each course in the course list until the matching course code is found. If the course exists, the system will display the complete course attributes such as the course code, course name, credit hour, summary, Teams link, and course type.

To implement this function, the system prompts the user to input the course code. The program then searches through the courses stored in the array. If a match is found, the course information will be displayed. If the course cannot be found in the list, the system will display a message saying “Course not found!” to inform the user that the search was unsuccessful.

The search function allows users to find a course by course code.

1. Implement linear search to find a course by course code
2. Input search criteria
3. Display full course attributes if found
4. Display "Course not found" if not found

Code

Linear Search

```
public int findCourseIndex(String code) {

    for(int i = 0; i < courseCount; i++) {

        if(courses[i].getCourseCode().equalsIgnoreCase(code)) {
            return i; // course found
        }

    }

    return -1; // course not found
}
```

```java
public void searchCourse() {

    System.out.print("Enter course code to search: ");
    String code = sc.nextLine();

    int idx = findCourseIndex(code);

    if(idx != -1) {
        courses[idx].displayCourse(); // successful search
    }
    else {
        System.out.println("Course not found!");
    }

}
```

```java
public void displayCourse() {

    System.out.println("==================================");
    System.out.printf("Course Code : %s%n", courseCode);
    System.out.printf("Course Name : %s%n", courseName);
    System.out.printf("Credit Hour : %d%n", creditHour);
    System.out.printf("Summary     : %s%n", summary);
    System.out.printf("Teams Link  : %s%n", teamsLink);
    System.out.printf("Type        : %s%n", courseType);
    System.out.println("==================================");

}
```

Output

```
Enter course code to search: CSEB5223
```

```
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
```

```
Enter course code to search: CSEB9999
```

```
Course not found!
```

9.4 Edit Function Development

With the Edit Function, the system first performs a search to find the target course that the user wants to edit. The user is required to enter the course code, and the system will search through the course list to locate the matching course. If the course is found, the system will display the complete course attributes. All attributes such as course name, credit hour, summary, Teams link, and course type can be edited except for the course code.

To implement this method, the system prompts the user to enter the course code. Once the course is found, the current course information will be displayed, and the user will be allowed to enter new values to update the course details. After the editing process is completed, the system will display the updated course information to validate the changes. If the course cannot be found, the system will display a message saying “Course not found!”.

Users can edit course details except for the course code.

a. Implement search function to find course

```
int idx = findCourseIndex(code);
```

b. Display complete attributes and allow editing except course code

Editable attributes:

- courseName
- creditHour
- summary
- teamsLink
- courseType

c. Display “Course not found”

```
if(idx == -1) {
    System.out.println("Course not found!");
}
```

d. Display edited course

```
c.displayCourse();
```

Output: Successful Edit

```
Enter course code to edit: CSEB4113
```

```
Editing course: CSEB4113
New Name (Software Quality): Advanced Software Quality
New Credit Hour (3): 4
New Summary (Learn software quality and testing.): Software testing and QA practices
New Teams Link (https://teams.microsoft.com/l/course/CSEB4113):
New Type (core): core
Course updated successfully!
```

Updated

```
==================================
Course Code : CSEB4113
Course Name : Advanced Software Quality
Credit Hour : 4
Summary     : Software testing and QA practices
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
```

Output: Course Not Found

```
Enter course code to edit: CSEB9999
```

```
Course not found!
```

9.5 Delete Function Development

Users can delete a course safely with confirmation.

Implementation

With the Delete Function, we needed to first implement the Search Function to find the course. It needed to display the complete course attributes for a successful search before sending a confirmation message to confirm the deletion. However, if the course wasn't able to be found, a message saying "Course not found" would be displayed. Otherwise, the search would display all the courses to validate the deletion of the selected course.

To create this method, we used the searchCourse method to find the selected course that we wanted to delete, by entering the course code. Once found, the system would display the attributes of the selected course, before giving the message to confirm its deletion. If you confirm it, the course is deleted and a message saying "Course deleted successfully!" would be displayed. The system would then display the rest of the courses. If you chose not to confirm it, a message saying "Deletion canceled." will be displayed. If the course is unable to be found, a message saying "Course not found!" will be displayed.

Code

```
public void deleteCourse() {
    System.out.print("Enter course code to delete: ");
    int idx = findCourseIndex(sc.nextLine());
    if(idx != -1) {
        courses[idx].displayCourse();
        System.out.print("Confirm delete? (y/n): ");
        if(sc.nextLine().equalsIgnoreCase("y")) {
            for(int i=idx;i<courseCount-1;i++) courses[i]=courses[i+1];
            courseCount--;
            // Remove relationships
            for(int s=0;s<studentCount;s++) relationship[s][idx]=false;
            System.out.println("Course deleted successfully!");
            viewAllCourses();
        } else System.out.println("Deletion canceled.");
    } else System.out.println("Course not found!");
}
```

Output

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
c
Enter course code to delete: CSEB5223
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
Confirm delete? (y/n): n
Deletion canceled.
```

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
c
Enter course code to delete: CSEB5223
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
Confirm delete? (y/n): y
```

```
Course deleted successfully!
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
==================================
Course Code : CCSB5113
Course Name : Software Security
Credit Hour : 3
Summary     : Fundamentals of software security.
Teams Link  : https://teams.microsoft.com/l/course/CCSB5113
Type        : elective
==================================
```

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
c
Enter course code to delete: CSEB5223
Course not found!
```

9.6 Data Display Function Development

The viewAllCourses() method displays all courses in the system.

Implementation

With the View All Function, we had to make a method when selected, would display all the entered courses and their attributes in a clear and organized manner. This is an important feature that needs to be executed after the Search, Edit or Delete function has completed their instructions. This is necessary to show the result of any changes that occurred after these functions are executed. It also has the option to display itself without the Search, Edit and Delete Functions.

Code

```
public void viewAllCourses() {
    if(courseCount==0){ System.out.println("No courses."); return; }
    for(int i=0;i<courseCount;i++) courses[i].displayCourse();
}
```

Output

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice: 2
==================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
==================================
Course Code : CCSB5113
Course Name : Software Security
Credit Hour : 3
Summary     : Fundamentals of software security.
Teams Link  : https://teams.microsoft.com/l/course/CCSB5113
Type        : elective
==================================
```

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
a
Enter course code to search: CSEB5223
===================================
Course Code : CSEB5223
Course Name : Software Construction & Methods
Credit Hour : 3
Summary     : Practice coding and build SLMS project.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
===================================
```

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
b
Enter course code to edit: CSEB5223
Editing course: CSEB5223
New Name (Software Construction & Methods): SCM
New Credit (3): 2
New Summary (Practice coding and build SLMS project.): Design software architecture.
New Teams Link (https://teams.microsoft.com/l/course/CSEB5223):
New Type (core): core
Course updated successfully!
==================================
Course Code : CSEB5223
Course Name : SCM
Credit Hour : 2
Summary     : Design software architecture.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
```

```
===== SLMS MENU =====
1. Add Course
2. View All Courses
3. Search/Edit/Delete Course
4. Add Student
5. View All Students
6. Search/Edit/Delete Student
7. Assign Student to Course
8. List Courses of Student
9. List Students of Course
0. Exit
Enter choice: 3
a. Search Course
b. Edit Course
c. Delete Course
c
Enter course code to delete: CSEB5223
==================================
Course Code : CSEB5223
Course Name : SCM
Credit Hour : 2
Summary     : Design software architecture.
Teams Link  : https://teams.microsoft.com/l/course/CSEB5223
Type        : core
==================================
Confirm delete? (y/n): y
Course deleted successfully!
==================================
Course Code : CSEB4113
Course Name : Software Quality
Credit Hour : 3
Summary     : Learn software quality and testing.
Teams Link  : https://teams.microsoft.com/l/course/CSEB4113
Type        : core
==================================
```

```
==================================
Course Code : CCSB5113
Course Name : Software Security
Credit Hour : 3
Summary     : Fundamentals of software security.
Teams Link  : https://teams.microsoft.com/l/course/CCSB5113
Type        : elective
==================================
```

10.0 References

- Moodle Development. (n.d.). MoodleDev: Moodle developer documentation. https://moodledev.io/
- Moodle. (n.d.). Moodle: The open-source learning platform (Source code). GitHub. https://github.com/moodle/moodle
- SourceMaking. (n.d.). SourceMaking: Software design patterns, refactoring, and UML examples. https://sourcemaking.com/
- GitByBit. (n.d.). GitByBit: Git tutorials, tips, and guides. https://gitbybit.com/
- draw.io. (n.d.). *draw.io: Online diagram software.* https://www.drawio.com/